

Member-only story

# How to Use lat.md: Turn Any Folder Into a Validated Knowledge Graph

A step-by-step guide to using lat.md, the open-source tool that builds a queryable knowledge graph with referential integrity.



Ana Bildea, PhD

Follow

6 min read · May 14, 2026



903



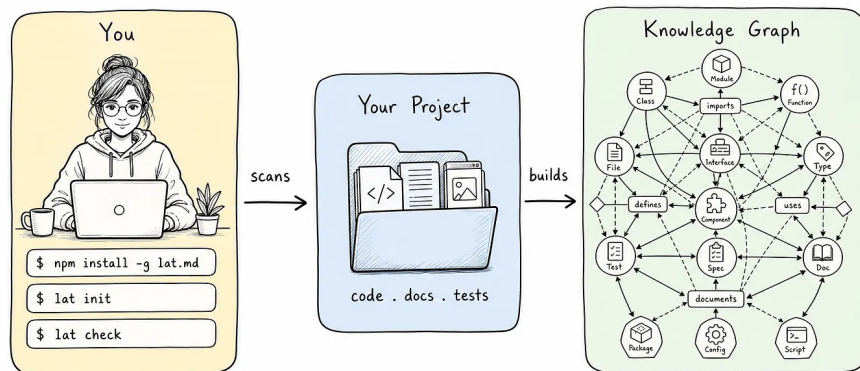
7



8



## LAT.MD: VALIDATED KNOWLEDGE GRAPHS



As AI becomes a bigger part of how we write code, developers are hitting a common problem: AI models can only “see” a small part of a project at once. When projects get large, simply giving the AI all your files doesn’t work well. It uses up too many resources and often leads to the AI getting confused or missing the big picture.

In my previous article, I explored how **Graphify** addresses this by building a persistent knowledge graph from your code, documents, and even media files like video and audio. If you haven’t read it yet, you can find the full guide on [how Graphify reduces token usage by up to 71.5x here](#).

Top highlight

While Graphify excels at high-level extraction, `lat.md` (Agent Lattice) provides a specialized solution for developers requiring rigorous validation and seamless integration with their existing workflows. Rather than merely mapping information, `lat.md` enforces referential integrity, ensuring that every documented concept remains synchronized with the underlying implementation.

## Why Simple Docs Fail as Projects Grow

A single “Read Me” file might work for a small project. But as a project grows to thousands of files with multiple teams involved, this simple approach breaks down in three ways.

First, the file becomes too long for the AI to read all at once. Second, the information in the file often becomes outdated because there’s no way to check if it still matches the actual code. Third, simple docs often list what the code does but forget to explain why it was written that way or what rules it must follow.

Q 1

`lat.md` solves these issues by breaking documentation into smaller, connected pieces and providing a tool to automatically check that everything stays in sync with your code.

## Step 1: Set Up Your Map

Starting with `lat.md` is easy and doesn’t require any complex setup. You just install it and run a single command in your project folder.

The setup process is interactive. it asks which AI tools you use (like Claude or Cursor) and helps you set up basic rules. It creates a special folder for your

project's "map" and adds instructions for your AI tools. These instructions tell the AI to look at the map instead of trying to read every file from scratch.

## **Step 2: Scanning Your Code**

The first thing `lat.md` does is scan your code locally on your computer. No code is sent to the internet during this step, keeping your work private.

The tool can read over 20 different programming languages. It identifies the main parts of your code, like functions and classes, and how they connect. Because this is done automatically, you can be sure the map accurately reflects your actual code.

### Step 3: Connecting Docs and Code

This is where `lat.md` is different from regular documentation. It lets you link your written notes directly to specific parts of your code.

Even better, you can add special comments in your code that point back to your notes. `lat.md` has a “check” command that looks at these links. If a link is broken or a piece of code is missing its documentation, the tool will let you know. This ensures that your notes and your code always tell the same story.

### Step 4: The “Summary First” Rule

To keep the map easy to use, `lat.md` requires every section of your notes to start with a short summary. This summary should be no more than a few sentences.

This isn’t just for show. When an AI is searching your map, it uses these summaries to quickly decide which parts are relevant. If a section is missing its summary, the “check” command will flag it. This keeps your documentation high-quality and easy for both humans and AI to navigate.

### Step 5: Automatic Checking

Documentation often gets ignored because it’s hard to keep up to date.

`lat.md` fixes this by making documentation checks part of your regular development process.

You can set it up so that every time you save new code, `lat.md` automatically checks for broken links or missing notes. If something is wrong, it prevents the code from being saved until it's fixed. This keeps your project's map from ever getting "stale" or incorrect.

## Getting Started and What to Expect

You don't have to document your whole project at once. Start with the most important parts, like how users log in or how data is saved. Run the setup, have your AI help write the first few notes, and then turn on the automatic checks.

Keep in mind that `lat.md` is still a growing project. While it's very fast and works with many languages, it might occasionally have trouble with extremely complex code. However, for most projects, it provides a level of organization and accuracy that traditional notes just can't match.

## Take Aways

`lat.md` is a big step forward in how we work with AI. It moves us away from only "searching" for information and targets organized and validated map of our projects.

The main idea is simple: by connecting our notes directly to our code and checking them automatically, we make our AI tools much smarter and more

reliable.

The map is how the AI understands your project. Everything else is just guessing.

I've seen many teams struggle with documentation that no one reads or updates. `lat.md` changes this by making documentation a useful, active part of the project. It's the first tool I've seen that makes the AI a partner in keeping the project's knowledge organized.

The next generation of tools will give AI better data. Tools like `lat.md` that organize and verify information will be essential for any team working with AI at scale. Data is messy generally and hard to keep it synchronised.

To try it yourself, install the package and run the setup in any project folder. You'll be surprised at how much a good map can improve your workflow. You improve your Agentic SDLC workflow only by testing and learning. Enjoy.

Thank you for reading. See you in the next one.

. . .

If this was useful, the clap button helps more people find it .

I write about agentic AI governance, agent architecture, optimisation and the infrastructure decisions that separate production systems from demos → [Subscribe](#)

Deploying long-running agents in a regulated environment?  
Let's talk → [LinkedIn](#)

Artificial Intelligence

Software Development

AI Agent

Programming

Knowledge Graph



**Published in Agentic Builders**

548 followers · Last published May 14, 2026

Follow

Agentic Builders is a publication dedicated to practitioners designing and deploying autonomous AI agents and multi-agent systems. It shares practical insights, architectures, experiments, and real-world use cases to help builders.



**Written by Ana Bildea, PhD**

2.2K followers · 411 following

Follow



Tech Lead AI | Follow for AI advice, step-by-step tutorials, tips, and end-to-end implementation in production - let's explore AI together!

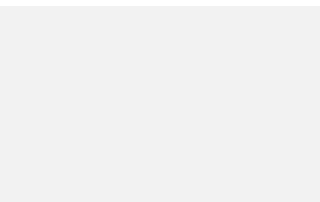
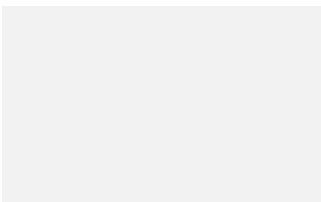


[Placeholder text for article title]

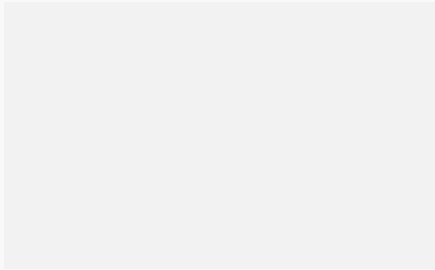
[Placeholder text for article description]

[Placeholder text for article content]

[Placeholder text for article content]

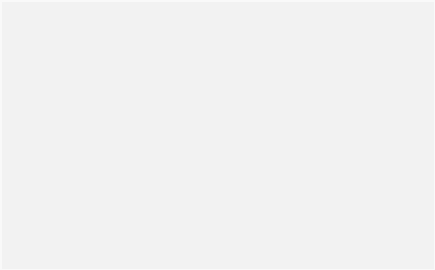


[Placeholder text for article content]



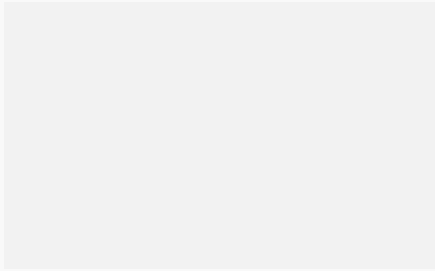
Text placeholder for the top-left content block.

Text placeholder for the top-left content block.



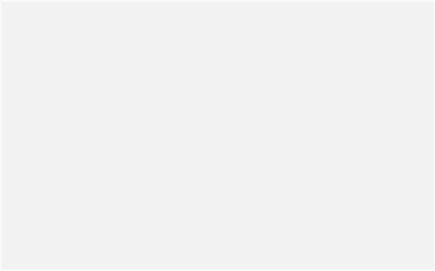
Text placeholder for the top-right content block.

Text placeholder for the top-right content block.



Text placeholder for the bottom-left content block.

Text placeholder for the bottom-left content block.



Text placeholder for the bottom-right content block.

Text placeholder for the bottom-right content block.